

Lock Free Concurrency

```
CTask Counter d  
int Count = 0;  
void increment() {  
    Count++;  
}
```

- ① Read Memory
- ② Increment by 1
- ③ Update Memory

Non Atomic

```
temp = Count;  
temp = temp + 1;  
Count = temp;
```

Atomic variables

operation → Single Unit
All or nothing.



```
int x = 4;
```

4
X

Atomic Integer

```
Count++
```

Non-atomic operation

Methods

- ↳ get()
- ↳ set()
- ↳ incrementAndGet()
 - ↳ $(++x)$
- ↳ getAndIncrement()
 - ↳ $x(x++)$
- ↳ decrementAndGet() $\rightarrow (--x)$
- ↳ getAndDecrement() $\rightarrow (x--)$
- ↳ addAndGet(value) $(: x = x + y)$
- ↳ getAndAdd(value)

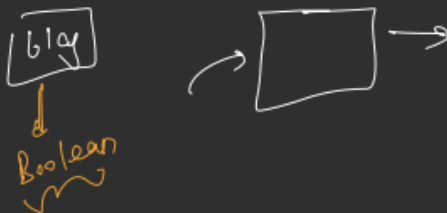
Count.get()

Count.set(4);

AtomicLong

AtomicBoolean

↳ producer-consumer problem



Synchronized ?

↳ Overhead

↳ locking Mechanism

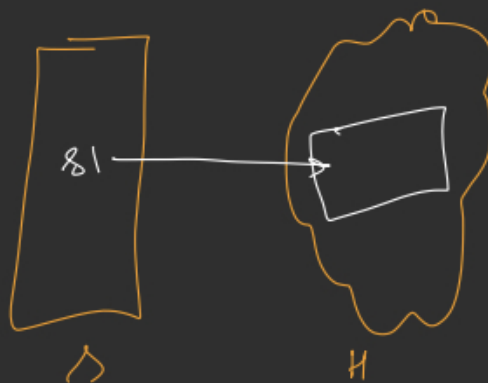
AtomicInteger Count = new AtomicInteger(4);

Count.incrementAndGet()

```
if (Count.get() > 4) {  
    Count.incrementAndGet();  
}
```

Atomic Reference

Student s1 = new Student();



Seat Booking

Class SeatBooking {

String seat = new String("EMPTY");

boolean bookSeat(String person) {

if (seat.equals("EMPTY")) {
 seat = new String(person);
 return true;

return false;

Class SeatBooking {

AtomicReference<String> seat =
 new AtomicReference<>("EMPTY");

boolean bookSeat(String name) {

String current = seat.get();

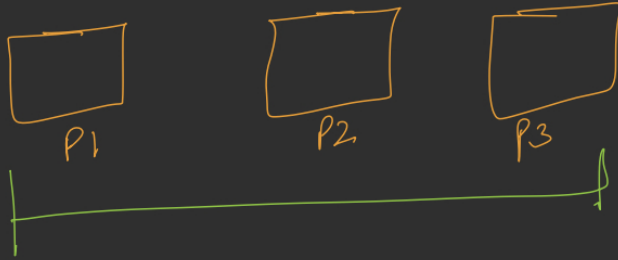
if (current.equals("EMPTY") == false) {
 return false;

return seat.compareAndSet("EMPTY", name);

① lock Signal

↳ Compare And Set (—, —)

x86 → CMPXCHG → lock



② Memory Controller

